

Lesson 7.1 — Production Polish

Lesson 7.1 — Production Polish

The features work. Now make the app robust. This lesson is the bag of small, high-leverage improvements that separate “demo-quality” from “ship-quality.”

By the end of this lesson you will:

- Add **accessibility** to the standings table, tab bar, and player navigation — the parts that block screen-reader users today.
- Implement a **React error boundary** so one broken component doesn't kill the whole page.
- Use Next.js **loading.tsx** for instant skeleton UIs.
- Audit your bundle size with **next build** and read the output critically.
- Run **Lighthouse** and act on its three highest-leverage suggestions.

You'll write maybe 60 lines of code in this lesson. The conceptual change is bigger: you start *seeing* the rough edges that beginners blow past.

1. The mindset shift

Up to now you've been building features. Each lesson asked: *“What does the user need next?”* and you answered with components.

Production polish is a different mindset. The questions are:

- *What happens when this fails?*
- *Who can't use this right now?*
- *How long does this take to load?*
- *What does the URL bar do when something crashes?*

These questions don't add new features. They harden the features you already shipped. **Junior engineers ship features. Senior engineers ship resilient features.**

If you do nothing else from this lesson, internalize that mindset shift. The rest is mechanics.

2. Accessibility (a11y) — the part that gets you hired

If you Google "React accessibility," you'll drown in 90-page documents. Most teams don't need 90 pages. They need **five rules** applied consistently. Those rules:

2.1. Use semantic HTML, not `<div>` soup.

A `<button>` is not a `<div onClick={...}>`. The browser gives you keyboard support, focus rings, and screen-reader announcements *for free* when you use the right element.

In our app, we already use `<button>` for the tab bar. Good. But the **round selectors in MatchesTab** were styled `<div>`s in some early drafts. If you find any, swap them to `<button type="button">`.

2.2. Every image must have alt text.

We don't use `<img>` heavily — most icons come from `lucide-react` (which renders SVG with `aria-hidden` by default). But if you ever add ``, write `alt="Trophy"` next to it. Decorative images get `alt=""` (empty string, not missing).

2.3. Tab interfaces need `role="tablist"`, `role="tab"`, `role="tabpanel"`.

Open `components/SnookerFantasyLeague.tsx`. Find the tab nav. Right now it looks like:

```
<nav style={{ ... }}>
  {tabs.map((tab) => (
    <button key={tab.id} onClick={() => setActiveTab(tab.id)}>
      {tab.label}
    </button>
  )
  )}
```

```
    )})  
</nav>
```

Upgrade to:

```
<nav role="tablist" aria-label="App sections" style={{ ... }}>  
  {tabs.map((tab) => (  
    <button  
      key={tab.id}  
      role="tab"  
      aria-selected={activeTab === tab.id}  
      aria-controls={`panel-${tab.id}`}  
      id={`tab-${tab.id}`}  
      onClick={() => setActiveTab(tab.id)}  
    >  
      {tab.label}  
    </button>  
  )})  
</nav>
```

And wrap the tab body:

```
<main role="tabpanel" id={`panel-${activeTab}`} aria-labelledby={`tab-${activeTab}`}>  
  {/* tab content */}  
</main>
```

Why bother? A screen reader user lands on your page. Without these attributes, they hear: *"button, button, button, button."* They have no idea those buttons are tabs.

With these attributes, they hear: *"App sections tab list, Standings tab, selected, 1 of 5."* That's the entire app surfacing in audio.

Why isn't this automatic? Because HTML can't read your design. The browser doesn't know that those four buttons are *related* and *mutually exclusive*. ARIA is how you tell it.

2.4. Every interactive element must be keyboard-reachable.

Try this: open your app in a browser and press Tab repeatedly. Can you get to every button? Every link? Every team row in the standings?

If not, you've got a problem. Common culprits:

- A `<div onClick={...}>` (not focusable). Fix: use `<button>` or add `tabIndex={0}` and an `onKeyDown` handler.
- A custom dropdown that traps focus. Fix: handle Escape to close, Tab to move out.

In our app, the **player cards in PlayersTab** are buttons (good). The **team selector in PredictionsTab** is a button (good). The **standings rows** — currently — are not clickable. If we ever made them clickable, they'd need to be `<button>` elements, not `<tr onClick={...}>`.

2.5. Color contrast: text must be readable.

Open Chrome DevTools ☞ Lighthouse ☞ Accessibility audit. It will flag any text whose contrast ratio drops below **4.5:1** (3:1 for large text).

The dark green #0F5132 on white passes. The yellow #FBBF24 on dark green passes. The light gray #9CA3AF on white *barely* passes for medium text. If you ever have to lighten text further for design reasons, push it to a heavier weight or a larger size to compensate.

2.6. The five-rule checklist

Run through this before shipping:

- ☐ **Semantic elements:** every interactive thing is a `<button>`, `<a>`, `<input>`, etc.
- ☐ **Alt text:** every `<img>` has one.
- ☐ **ARIA roles:** tab interfaces, dialogs, alerts, and lists are correctly labeled.
- ☐ **Keyboard reachable:** Tab walks through every interactive element.
- ☐ **Color contrast:** Lighthouse a11y audit passes.

This checklist is the answer to “*How do you handle accessibility on your team?*” in an interview. Memorize it.

3. Error boundaries — when one component crashes

By default, **a single thrown error inside a React component takes down the entire tree**. The whole page goes white. Users see nothing.

That's a terrible UX. Worse, it's a *silent* failure in production — your error tracker (Sentry, Datadog, etc.) might catch it, but the user just sees a broken page with no explanation.

Error boundaries fix this.

3.1. The class-component reality

Error boundaries are one of the **two places React still requires a class component** (the other is the `componentDidCatch` lifecycle that error boundaries themselves use). There is no hook for this. Yet.

Create `components/ErrorBoundary.tsx`:

```
'use client';

import { Component, type ReactNode } from 'react';

interface Props {
  children: ReactNode;
  fallback?: ReactNode;
}

interface State {
  hasError: boolean;
  error?: Error;
}

export default class ErrorBoundary extends Component<Props, State> {
  constructor(props: Props) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError(error: Error): State {
```

```

    return { hasError: true, error };
  }

  componentDidCatch(error: Error, info: { componentStack: string }) {
    console.error('Error boundary caught:', error, info.componentStack);
    // In production, send to Sentry / Datadog / etc.
  }

  render() {
    if (this.state.hasError) {
      return (
        this.props.fallback ?? (
          <div
            role="alert"
            style={{
              padding: 24,
              background: '#FEE2E2',
              border: '2px solid #DC2626',
              borderRadius: 12,
              color: '#7F1D1D',
            }}
          >
            <strong>Something went wrong.</strong>
            <div style={{ marginTop: 8, fontSize: 13 }}>
              {this.state.error?.message ?? 'Unknown error'}
            </div>
            <button
              onClick={() => this.setState({ hasError: false, error: undefined })}
              style={{ marginTop: 12, padding: '6px 14px', borderRadius: 6 }}
            >
              Try again
            </button>
          </div>
        )
      );
    }
  }

```

```

    }
    return this.props.children;
  }
}

```

3.2. Wrap things that might break

In `app/page.tsx`:

```

import ErrorBoundary from '@components/ErrorBoundary';
import SnookerFantasyLeague from '@components/SnookerFantasyLeague';

export default function Page() {
  return (
    <ErrorBoundary>
      <SnookerFantasyLeague />
    </ErrorBoundary>
  );
}

```

For finer-grained control, wrap individual tabs:

```

<ErrorBoundary fallback={<div>Analytics failed to load.</div>}>
  <AnalyticsTab teams={teamsWithScores} />
</ErrorBoundary>

```

3.3. The interview question this answers

Q: “How do you handle errors in a React app?”

A: “Three layers. (1) Error boundaries around any component subtree that could crash, with a friendly fallback UI and a logger that ships the error to Sentry. (2) Try/catch around any async work that could reject — fetch failures, JSON parse errors. (3) Defensive null checks in the render path for any data that might be missing. Error boundaries don’t catch errors in event handlers or async code, so the three layers complement each other.”

That’s a senior-grade answer. It hits the **what** (boundaries, try/catch, null checks), the **why**

(boundaries don't catch async), and the **so what** (Sentry).

4. Loading states with `loading.tsx`

Next.js App Router has a built-in primitive for “show this while a route segment is loading.”

Create `app/loading.tsx`:

```
export default function Loading() {
  return (
    <div
      role="status"
      aria-live="polite"
      style={{
        minHeight: '60vh',
        display: 'grid',
        placeItems: 'center',
        fontFamily: "'Roboto', sans-serif",
      }}
    >
      <div>
        <div
          style={{
            width: 60,
            height: 60,
            border: '4px solid #FEF3C7',
            borderTopColor: '#0F5132',
            borderRadius: '50%',
            animation: 'spin 0.8s linear infinite',
            margin: '0 auto 16px',
          }}
        />
        <div style={{ color: '#6B7280', fontWeight: 600 }}>Loading league da
      </div>
      <style>{`
        @keyframes spin {
```

```

        to { transform: rotate(360deg); }
      }
    `}</style>
  </div>
);
}

```

4.1. When does this fire?

Whenever a route's server component is doing async work (`await`, dynamic imports). Right now our app is fully static — all data is in `lib/`. So `loading.tsx` will rarely fire in this app.

That's still worth setting up. The day you migrate to a real backend (fetching tournament data from Sportradar's API, say), the loading state is already in place. Future-you will thank present-you.

4.2. Why `aria-live="polite"`?

Screen readers announce the loading state to the user, but in a non-disruptive way. `aria-live="assertive"` interrupts whatever the user is hearing — too aggressive for a spinner.

5. Bundle audit with `next build`

Run:

```
npm run build
```

You'll see output like:

Route (app)	Size	First Load JS
└─ ○ /	312 kB	407 kB
└─ ○ /_not-found	872 B	88 kB
+ First Load JS shared by all	87.4 kB	

5.1. What to look at

- **Size** — the JS for *just this route's* code.
- **First Load JS** — the JS the browser downloads when it first hits this route, including shared chunks.
- **Shared chunks** — code split out so that navigating between routes doesn't re-download it.

5.2. The numbers to worry about

For a public-facing app, **First Load JS under 200 KB** is excellent. **Under 500 KB** is fine. **Over 1 MB** is a red flag.

Our app is heavy because **Recharts is heavy** (~150 KB minified, gzipped). That's a known trade-off — we picked Recharts for ease of use, knowing it costs us bundle size. If we ever needed to slim this down:

1. **Dynamic-import the Analytics tab** so its bundle only loads when the user clicks it.
2. **Switch to a lighter charting library** (Visx, custom SVG).
3. **Server-render the charts** (Recharts has experimental SSR; not always worth it for fantasy leagues).

You don't have to do this today. You *do* have to **know to do it** when an interviewer asks.

5.3. Dynamic import in 30 seconds

In components/SnookerFantasyLeague.tsx:

```
import dynamic from 'next/dynamic';

const AnalyticsTab = dynamic(() => import('./tabs/AnalyticsTab'), {
  loading: () => <div>Loading charts...</div>,
  ssr: false,
});
```

The component is loaded *only when it first renders*. The Recharts JS is no longer in your initial bundle.

This is a one-line optimization. Use it sparingly — it adds a network round-trip when the user finally clicks the tab. The trade-off is worth it for heavy components used by a minority

of visitors.

6. Lighthouse — the report card

Open Chrome DevTools ☞ Lighthouse ☞ click *Analyze page load*. You'll get four scores:

- **Performance** — how fast does the page render?
- **Accessibility** — did you follow the rules?
- **Best Practices** — HTTPS, no console errors, etc.
- **SEO** — meta tags, mobile-friendly, etc.

6.1. The three highest-leverage fixes

Almost every React app has the same three issues on the first Lighthouse run:

1. **Missing meta description.** Add one in `app/layout.tsx`:

```
export const metadata: Metadata = {  
  title: 'Snooker Fantasy League',  
  description: 'Live standings, picks, and analytics for our 8-team fant  
};
```

2. **Color contrast on a button.** Pick a darker text color on light backgrounds.
3. **Missing lang attribute.** Already in `app/layout.tsx` if you used the boilerplate (`<html lang="en">`). Verify it.

A 100/100 Lighthouse score is rare and not the goal. **A 90+ across the board** is the realistic target. Anything you can fix in 5 minutes and gain a point on, you should.

7. Performance: the React-specific wins

Lighthouse measures *page-load* performance. There's another category: *interaction* performance — how snappy the app feels after it's loaded.

7.1. The useMemo you already wrote

Remember useMemo from Chapter 4? You used it for teamsWithScores. That's the kind of thing that helps interaction performance. Without it, every re-render recomputes scores for every team. With it, that work is done once.

7.2. React.memo for expensive children

If you ever have a chart that re-renders on every parent state change, wrap its export:

```
export default React.memo(LeagueAgreementChart);
```

Now React only re-renders the chart when its **props** change. Don't reach for this until profiling shows you need it.

7.3. The React DevTools Profiler

Install React DevTools (browser extension). Open the Profiler tab. Click record. Click around the app. Stop recording. You'll see a flame graph of what re-rendered and how long it took.

This is the tool that turns "the app feels slow" into "the standings table re-renders 12 times when I switch tabs."

You don't have a perf problem in this app. You should still know how to use the profiler — it's an interview-grade skill.

8. Defensive rendering

Two patterns to internalize:

8.1. Guard against missing data

```
{team.scores ? <TeamCard team={team} /> : <div>No scores yet.</div>}
```

The && shortcut also works:

```
{team.scores && <TeamCard team={team} />}
```

But beware: `0 && <Foo />` renders `0`, not nothing. Use `team.scores !== null && <TeamCard />` if you might have a numeric-zero value.

8.2. Empty states

When an array is empty, **render an empty state, not a blank gap**:

```
{matches.length === 0 ? (  
  <div role="status">No matches yet for this round.</div>  
) : (  
  matches.map(/* ... */)   
)}
```

Empty states are the difference between a broken-looking app and a polished one. Apply this everywhere a list might be empty.

9. Vibe prompt you would have used

“Audit my Next.js 14 app for production polish. Specifically: (1) add `role="tablist"`, `role="tab"`, `role="tabpanel"`, and `aria-selected` to my tab bar. (2) Generate a class-based `ErrorBoundary` component with a friendly fallback UI and a `componentDidCatch` that logs to console. Wrap the root component in it. (3) Add `app/loading.tsx` with a polite `aria-live` spinner. (4) Add a metadata export with title and description. Don't change my existing styling — just layer the a11y and resilience on top.”

That prompt is **specific**. It calls out exactly which ARIA attributes, which React feature, which file paths. The LLM produces clean code instead of generic boilerplate.

10. CHECK YOURSELF

Don't proceed until you can explain each of these:

- ☐ Why does a tab bar need `role="tablist"` and `role="tab"` and `role="tabpanel"`?

- ☐ What's the difference between `aria-live="polite"` and `aria-live="assertive"`?
- ☐ Why is an error boundary still a class component in 2024?
- ☐ What does `getDerivedStateFromError` do that `componentDidCatch` doesn't?
- ☐ Why doesn't an error boundary catch errors thrown in event handlers?
- ☐ What's the practical effect of wrapping a tab in `dynamic(() => import(...), { ssr: false })`?
- ☐ What contrast ratio does Lighthouse demand for body text?
- ☐ When would you use `React.memo`?
- ☐ What's the danger of something `&& <Foo />` when something could be 0?
- ☐ Why does the "five-rule a11y checklist" matter more than memorizing all of WCAG?

If any answer is fuzzy, scroll back. The next lesson covers deployment and testing — both of which assume your app is shippable.

11. Where you are now

Your app is now **resilient**:

- Screen-reader users can navigate it.
- A crash in one tab doesn't kill the page.
- Loading states are in place for the day you fetch live data.
- The bundle is reasonable; the heavy pieces are flagged for future optimization.
- Lighthouse won't embarrass you.

Move on to [02-deployment-and-testing.md](#).